

Meridian Data An AI-Powered NL-to-SQL Database Explorer

Internal

Project Guide:

Prof. Ashwini Mathur

Assistant Professor

School of Computer Science and Engineering

Team Members

1RVU23CSE093 — Ayon Aryan

1RVU23CSE180 — Hardik Goel

1RVU23CSE055 — Aniket Kumar

1RVU23CSE029 — Aditya Kumar

Contents

1. Title & Problem Definition
2. Introduction & Scope
3. Objectives
4. Literature Survey
5. Proposed Methodology
6. Requirement Specification
7. Design Approach & Architecture
8. Innovations
9. Implementation
10. Evaluation, Testing & Performance
11. Conclusion & Future Work
12. References

1. Title and Problem definition

- **Problem** — querying databases requires SQL fluency, a barrier for students, analysts and domain experts.
- **Existing systems** — single proprietary engine, closed-source, and send data to the cloud unconditionally.
- They hallucinate SQL and lack safety, authorization and human oversight.
- **Novelty** — hybrid deterministic + LLM querying over eight engines, with layered safety, human review and cloud-to-local privacy.

Introduction

- Converts plain-English requests into validated, dialect-specific SQL using LLMs given the live schema.
- **Scope** — NL querying, deterministic introspection, AI analysis with auto-charts, dashboards, and CSV / PowerPoint export.
- Connects to SQLite, PostgreSQL, MySQL, MSSQL, Oracle, MongoDB, Cassandra and Redis via one adapter layer.
- **Doubles as a productivity tool and a DBMS teaching aid** — it shows and explains the SQL it generates.

Objectives

- Convert natural language into validated, dialect-aware SQL via a schema-aware prompt.
- Answer common introspection commands deterministically, without the LLM.
- Enforce two-layer safety with role-based access control.
- **Keep a human in the loop for writes** — dry-run, snapshot and rollback.
- Provide AI analysis, auto chart selection, dashboards and export.
- Support cloud (Groq) and local (Ollama) models with automatic failover.

Literature Survey

Evolved from rule-based NLIDBs to neural text-to-SQL (Seq2SQL / WikiSQL, SQLNet) and the Spider benchmark.

RAT-SQL and BRIDGE solved schema linking; PICARD added constrained decoding for valid SQL.

LLM era — in-context learning, DIN-SQL decomposition, DAIL-SQL prompting; BIRD shows real databases stay hard (GPT-4 ~55%).

Safety / RBAC and human-in-the-loop research inform the design.

Meridian Data integrates these validated components rather than proposing a new parser.

Literature Survey — Summary

Work	Technique	Key Limitation
Seq2SQL (2017)	RL over WikiSQL	Single-table queries only
SQLNet (2017)	Sketch + column attention	Tied to WikiSQL grammar
Spider (2018)	Cross-domain benchmark	Schema-only; clean DBs
PICARD (2021)	Constrained decoding	Syntactic, not semantic, validity
DIN-SQL (2023)	In-context decomposition	High token cost / latency
BIRD (2023)	Dirty-DB benchmark	GPT-4 only ~55% accuracy

Proposed Methodology

Intent classification — introspection answered deterministically; data questions go to the LLM.

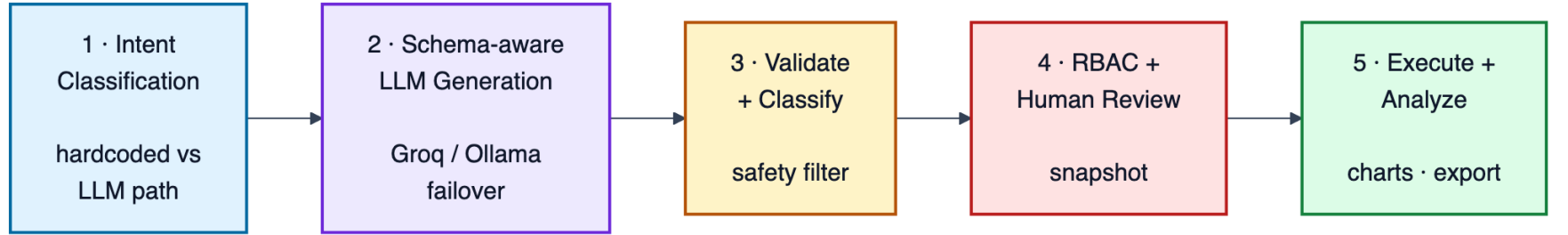
Schema-aware generation — schema (columns, keys, indexes, samples) injected into a dialect prompt; Groq / Ollama failover.

Validation & authorization — classify (READ / WRITE / SCHEMA / SYSTEM) → safety filter → role check.

Execution or review — READ runs paginated; WRITE / SCHEMA go to human review with snapshot.

Analysis & presentation — AI summary, auto-selected chart, CSV / PowerPoint export.

Proposed Methodology — Pipeline



Requirement Specification

Functional — NL→SQL generation; deterministic introspection; classification + safety; RBAC; human review with dry-run / snapshot; multi-DB connect; AI analysis, dashboards and export.

Non-functional — security: block DDL / injection, encrypt credentials (Fernet).

Reliability — introspection always works; automatic cloud↔local failover.

Performance — millisecond introspection, sub-second cloud generation.

Usability, privacy (local mode) and extensibility (new engine = new adapter).

Design approach

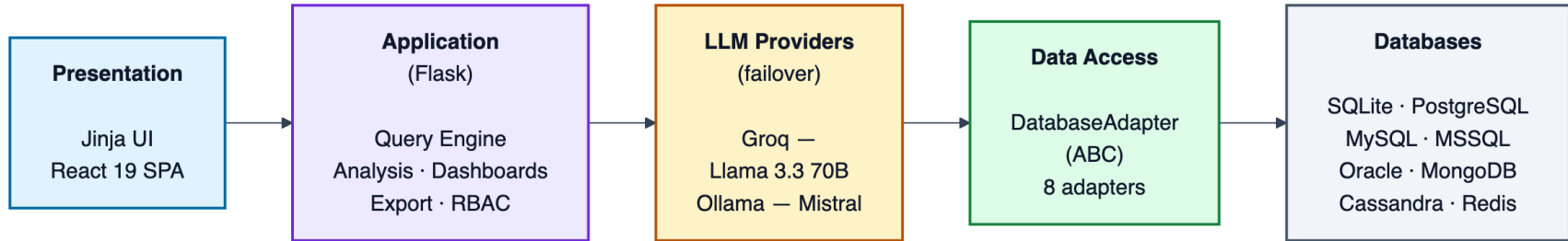
Layered — presentation → application engines → LLM provider (failover) → data-access (Adapter) → persistence.

Adapter pattern — one DatabaseAdapter interface (connect, get_schema, list_tables, execute) with eight concrete adapters; app code never imports a driver.

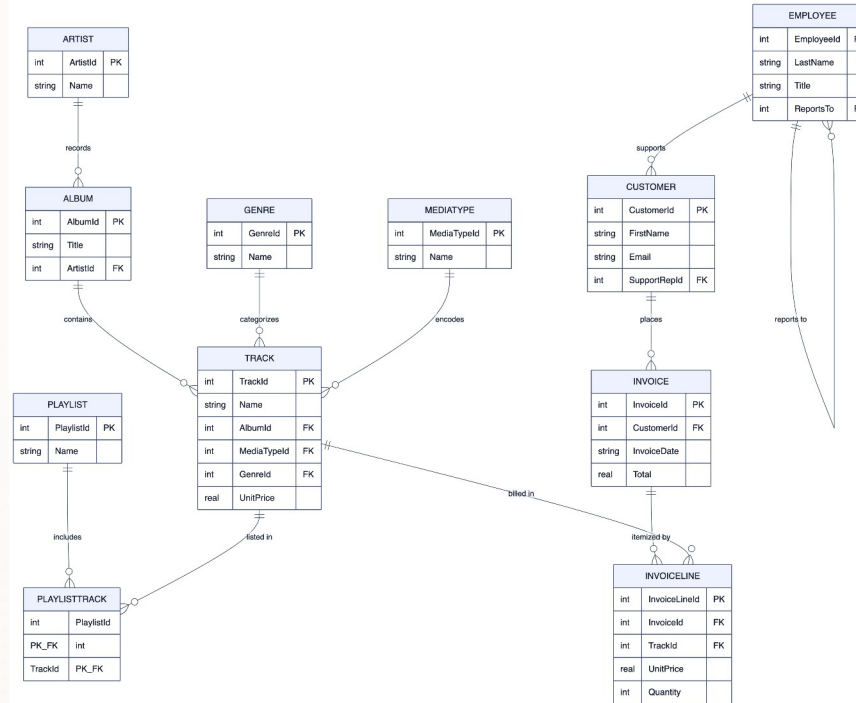
A two-layer guardrail (classification + safety) plus an explicit human gate isolate the LLM from the database.

Conversation context (last five turns) enables follow-up and query refinement.

Architecture diagram



Database Design — ER Diagram (Sample Schema)



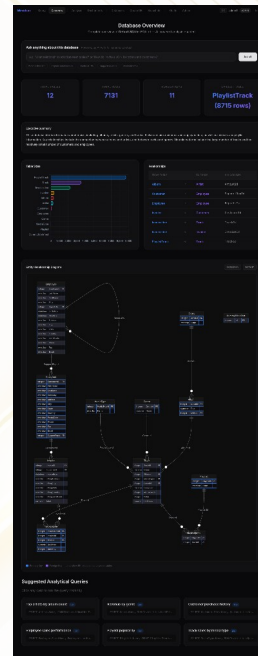
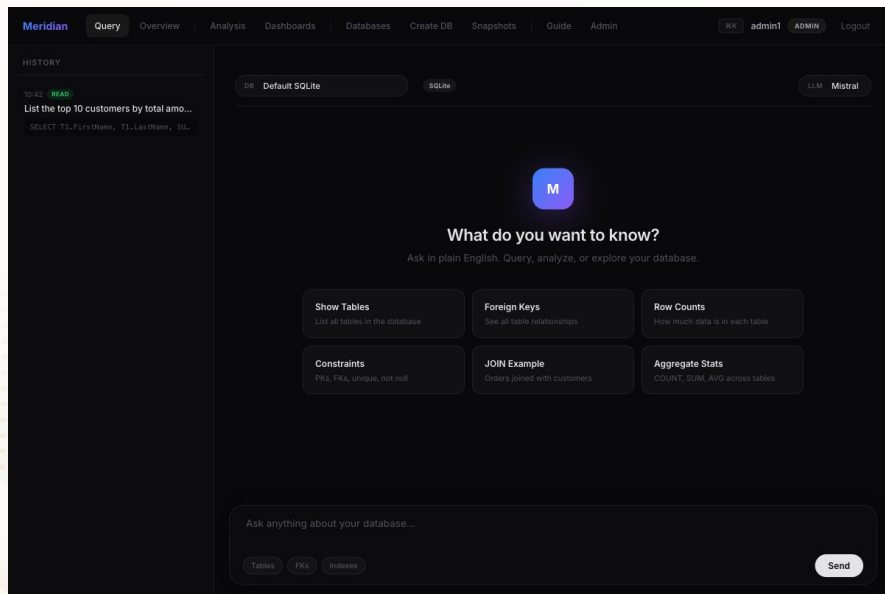
Innovations

- **Hybrid execution** — deterministic introspection + LLM generation: correctness where cheap, intelligence where needed.
- One adapter abstraction extends NL querying across eight relational and NoSQL engines (JSON contract for Mongo / Redis).
- **Cloud-to-local failover** — resilient and privacy-flexible.
- **Defence-in-depth** — the LLM is never trusted to be safe; a separate validator and human gate guard execution.

Implementation

- Python 3.11 + Flask backend; two front-ends (Jinja + React 19 SPA) over one JSON API.
- **LLM layer** — Groq SDK (Llama 3.3 70B) and local Ollama (Mistral) with per-dialect prompt templates.
- Validator, Fernet-encrypted connection manager, and a per-engine snapshot / rollback engine.
- AI analysis via Groq JSON mode; dashboards persisted as JSON; PowerPoint export via python-pptx.

Implementation — Working Application



Evaluation Metrics

Functional correctness — each feature works on the sample databases.

Generation correctness — generated SQL executes and returns the intended rows.

Safety — destructive and unauthorized statements are blocked.

Latency — time to answer, by execution path and provider.

Token efficiency — prompt and completion tokens per call.

Testing & Validation

Unit — hardcoded commands (show tables = 12, describe, foreign keys) and the validator: all pass in < 20 ms.

Integration — generation → validation → authorization → execution; write-review → snapshot → execute; failover: all pass.

System — end-to-end NL query, DB switching, AI analysis, dashboards, destructive-command blocking: all pass on both front-ends.

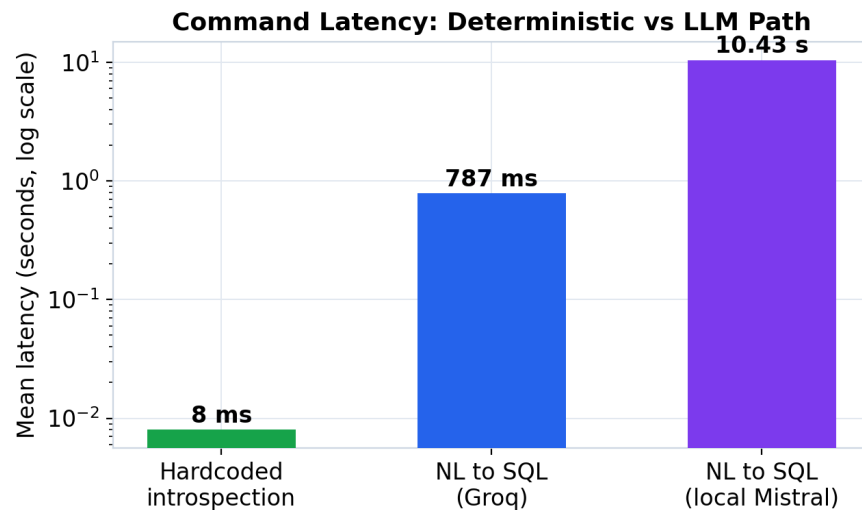
Performance Analysis

Groq cloud NL→SQL — mean 0.79 s, median 0.73 s (46 logged calls).

Local Mistral — ~10.4 s (fully private, on-device).

Hardcoded introspection — ~8 ms end-to-end (no LLM), ~100× faster.

Generated SQL validated against expected results on the Chinook DB.



Evaluation Results

100%

Unsafe queries
contained (12/12)

0.79s

Mean NL→SQL latency
(Groq cloud)

≈8 ms

Hardcoded introspection
(end-to-end, no LLM)

3/3

Legit reads allowed
(no false blocks)

Method — labelled NL→SQL queries on the Chinook DB (generated SQL executed and compared to gold result sets), a 12-prompt destructive / injection battery, and latency over 113 logged LLM calls.

Result — correct SQL across aggregate, projection, top-N, group-by and join queries; 100% of unsafe statements contained (10 blocked, 2 routed to human review) with no false blocks on valid reads.

Hardware and/ or System Requirement

Software — Python 3.11+ & Flask; Groq SDK / Ollama; DB drivers (psycopg2, PyMySQL, pymongo, redis, ...); cryptography; python-pptx; React / Vite / Tailwind.

Hardware — any modern browser; 4 GB RAM (cloud) or 16 GB + optional GPU (local); 2–10 GB storage.

Why — Flask is lightweight; Groq's LPU gives low latency; Ollama enables privacy; per-engine drivers give multi-database reach.

Conclusion

Meridian Data turns LLM text-to-SQL into a dependable tool — schema-aware prompting, deterministic shortcuts, layered validation, RBAC, human review and provider failover.

Achieves sub-second cloud generation and millisecond introspection, and blocks every destructive operation.

Serves as both an analyst productivity aid and a DBMS teaching instrument.

Future Work

Replace the keyword safety filter with a full SQL parser (e.g. sqlglot).

Unify the duplicated Jinja and JSON pipelines behind one service layer.

Add retrieval-augmented schema selection for very large schemas.

Self-correction — feed execution errors back to the model to repair queries.

Stronger credential security; a quantitative accuracy study on Spider and BIRD.

References

- [1] E. F. Codd, “A relational model of data for large shared data banks,” Comm. ACM, 1970.
- [2] T. Yu et al., “Spider: cross-domain text-to-SQL dataset,” EMNLP, 2018.
- [3] T. Scholak et al., “PICARD: constrained auto-regressive decoding,” EMNLP, 2021.
- [4] M. Pourreza, D. Rafiei, “DIN-SQL: decomposed in-context learning,” NeurIPS, 2023.
- [5] J. Li et al., “BIRD: a big bench for database-grounded text-to-SQL,” NeurIPS, 2023.
- [6] R. S. Sandhu et al., “Role-based access control models,” IEEE Computer, 1996.

Thank You